Latency-Aware Split Learning Optimization via Genetic Algorithms

Le Hoang Trung
Research Engineer at LG Electronics R&D
Da Nang, Vietnam
trunglehoang.117@gmail.com

Tan Y Nguyen*
Faculty of Engineering and Technology, University of Phan Thiet
Phan Thiet, Binh Thuan
Vietnam
nty@upt.edu.vn

Duy-Dong Le
University of Economics Ho Chi Minh City
Ho Chi Minh, Vietnam
dongld@ueh.edu.vn

Thai-Thinh Dang
University of Economics Ho Chi Minh City
Ho Chi Minh, Vietnam
thinhdt@ueh.edu.vn

Tran Anh Khoa
National Institute of Information and Communications Technology
Tokyo, Japan
tran@nict.go.jp

Abstract

With the rapid growth of edge computing and Internet of Things (IoT) applications, deploying deep neural networks (DNNs) on resource-constrained devices presents significant challenges due to limited computational power and network variability. Split Learning (SL) has emerged as an effective framework to address these challenges by enabling collaborative inference across edge-cloud systems. However, a critical issue in SL is determining the optimal split point to balance local computation and communication overhead, thereby minimizing end-to-end inference latency. Existing heuristic or exhaustive search approaches are inefficient and impractical for real-world devices with limited processing and network capabilities. To address this, we propose a Genetic Algorithm (GA)-based method that models split point selection as an optimization problem. The GA evolves candidate splits to minimize total latency using a fitness function based on measured computation and transmission delays. Experiments on large-scale models demonstrate that our approach achieves a 74.38% optimization efficiency, compared to only 28.70% achieved by brute-force baselines. By efficiently exploring the search space, the GA adapts to varying device constraints and network conditions, offering a scalable and practical solution for deploying deep learning in resource-limited environments.

Keywords

Split Learning, Deep Neural Networks, Resource-Constrained, Genetic Algorithm, Optimization.

*Corresponding author: Tan-Y Nguyen is a Ph.D student at Ton Duc Thang University, Vietnam (nty@upt.edu.vn or nguyentany.st@tdtu.edu.vn)

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICDAR'25, June 30-July 3, 2025, Chicago, IL, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1890-8/25/06
https://doi.org/10.1145/3733566.3734433

ACM Reference Format:
Le Hoang Trung, Tan Y Nguyen, Duy-Dong Le, Thai-Thinh Dang, and Tran Anh Khoa. 2025. Latency-Aware Split Learning Optimization via Genetic Algorithms. In *6th Workshop on Intelligent Cross-Data Analysis and Retrieval (ICDAR'25), June 30-July 3 2025, Chicago, IL, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3733566.3734433

1 Introduction

Federated Learning (FL) has become a mainstream approach for enabling distributed machine learning with strong privacy guarantees [12]. By allowing clients to collaboratively train models without transmitting raw data across edge devices, FL has found widespread applications in healthcare, finance, and edge intelligence [7–9]. However, traditional FL assumes that each client can locally train a full deep neural network (DNN), which becomes impractical as model sizes grow and client devices remain resource-constrained. For modern deep models, the local computational and memory requirements often exceed the capabilities of mobile, IoT, or embedded devices.

To address these limitations, Split Learning (SL) has been proposed as a natural extension of FL [6, 14]. In SL, the model is partitioned into two distinct segments: a client-side submodel and a server-side submodel. The client computes forward passes up to a designated split layer, transmits intermediate activations to the server, which then completes the forward and backward passes. This significantly reduces the computational and memory burden on client devices, making SL highly attractive for real-world edge AI deployments [15]. Moreover, SL facilitates large-scale client participation, supports training of complex models that cannot be handled entirely on-device, and inherently enhances data privacy since raw data and labels are never fully shared [11].

Despite these promising advantages, several critical challenges hinder the widespread adoption of Split Learning. First, the choice of the split point is typically manual or heuristic-based, neglecting dynamic system conditions such as variable bandwidth and heterogeneous device capabilities. Second, most existing studies apply SL to relatively moderate-sized models (e.g., VGG-16, shallow CNNs) and have not thoroughly explored its behavior on modern,

large-scale DNNs. Third, there has been limited research into systematic optimization of split points to minimize total inference latency by balancing client computation and server communication. As emphasized in recent surveys [11, 16], the automatic and adaptive optimization of split points remains an important but underexplored research area.

Preliminary works, such as Dynamic Split Computing [1], have attempted to dynamically adjust the split location based on network conditions. However, these methods typically rely on predefined search strategies rather than performing a comprehensive global optimization over the model architecture. Consequently, they struggle to generalize across diverse models and system environments, often resulting in suboptimal configurations, especially under highly dynamic operating conditions.

Motivated by these limitations, we propose a novel Genetic Algorithm (GA) [4]-based framework for automatic split point optimization in SL as shown in Figure 1. By formulating the split selection problem as an end-to-end latency minimization task, we leverage the global search capabilities of GAs to efficiently evolve candidate split configurations and identify optimal solutions. Our method is model-agnostic, dynamically adaptive, and applicable to complex architectures without requiring retraining or model-specific fine-tuning.

Through extensive experiments on large-scale models such as EfficientNet-B7 [13], we demonstrate that our GA-based approach achieves an optimization efficiency of 74.38%, significantly outperforming the baseline brute-force search (28.70%) and dynamic heuristic approaches. Furthermore, our framework naturally adapts to varying client/server computational capacities and fluctuating network bandwidths, making it highly practical for real-world Edge AI deployments.

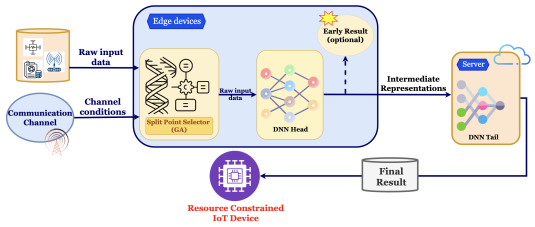


Figure 1: Overview of split learning approach.

In summary, the key contributions of this paper are:

- **A novel optimization method:** We introduce the first GA-based method for automatic split point selection in SL, framing it as a latency minimization problem and providing a scalable, model-independent solution.
- **Superior performance and generalization:** Our approach significantly outperforms existing heuristic and brute-force methods in optimization efficiency, validated across large-scale DNNs and under diverse system conditions.
- **Practical applicability for Edge AI:** By dynamically adapting to heterogeneous devices and changing network conditions without retraining, our method enables practical, scalable deployment of SL in real-world resource-constrained environments.

The remainder of this paper is organized as follows. Section 2 reviews the related work that forms the foundation of this study. Section 3 describes the proposed methodology in detail, including the system model, problem formulation, and optimization framework. Section III presents the experimental setup, performance evaluation, and analysis of the results. Finally, Section IV concludes the paper and discusses potential directions for future research.

2 Related Works

2.1 Dynamic Split Computing

Dynamic Split Computing has been proposed as a means to dynamically determine the optimal split point between client and server based on real-time network conditions [1]. By exploiting natural bottlenecks within DNN architectures, dynamic split computing achieves faster inference under fluctuating data rates and server loads without requiring model retraining or extensive hyperparameter tuning. Furthermore, it reduces server load and addresses privacy concerns by transmitting intermediate activations rather than raw inputs. However, the approach is largely heuristic-based and lacks a global optimization framework to systematically identify optimal splits across different models and scenarios.

2.2 Compression-Based Split Learning

To address the inefficiencies of naïve split computing approaches, supervised compression techniques have been introduced [11]. Supervised Compression for Split Computing (SC2) learns compressed intermediate representations optimized for downstream tasks, balancing computation, communication, and accuracy. The SC2 framework also proposes new evaluation metrics and benchmarks, providing a more holistic assessment of split computing strategies. Nevertheless, current compression-based approaches primarily focus on data size and model accuracy, while leaving computational resource constraints and dynamic adaptability relatively unexplored.

2.3 Energy-Aware Split Learning

In the context of IoT devices powered by energy harvesting, SplitFed Learning (SFL) introduces selective updating methods that balance energy availability with training objectives [2]. By performing selective activations/gradients updates based on energy and loss dynamics, significant energy savings can be achieved with minimal impact on model accuracy. Although highly effective in energy-constrained environments, these methods are tailored for specific energy profiles and do not generalize well to dynamic bandwidth conditions or heterogeneous client-server architectures.

2.4 Privacy and Scalability in Split Learning

While Split Learning inherently enhances privacy by avoiding raw data sharing, leakage through intermediate activations remains a concern. To mitigate this, the Multiple Activations and Labels Mix (MALM) method has been proposed [17]. MALM generates mixed activations and obfuscated labels to reduce their correlation with the raw data, thereby lowering the risk of reconstruction attacks. Although effective in improving privacy, such approaches introduce

additional communication and computation overhead, which can be critical for extremely resource-constrained edge devices.

SFL combines SL with FL, offering decentralized training without full model exposure [5]. Recent studies have analyzed how cut layer selection influences model convergence and performance in SFL settings. Results indicate that in SFL-V2 (shared server model), careful cut layer placement is essential for achieving superior accuracy, particularly under data heterogeneity. However, despite theoretical insights, practical, automated methods for optimal cut layer selection remain underdeveloped.

To address scalability challenges in Split Learning, adaptive split learning (AdaSplit) frameworks have been proposed [3]. AdaSplit dynamically adjusts the split point and communication strategies to reduce bandwidth consumption and improve performance across heterogeneous clients. Experimental results demonstrate AdaSplit's effectiveness under limited resources. However, it still relies on heuristic tuning, and its adaptability under highly dynamic and unpredictable environments remains to be validated.

With the growing vision of 6G networks supporting distributed edge intelligence, Split Learning has been identified as a key enabler for resource-efficient edge AI [10]. Recent studies propose architectures and resource management strategies for integrating Split Edge Learning (SEL) into wireless multi-edge systems. Nonetheless, challenges such as convergence guarantees, asynchronous learning, and efficient multi-edge collaboration continue to be open problems.

3 Methodology

3.1 Problem Formulation

We consider the deployment of a DNN over a client-server system under SL. Let the DNN be represented as:

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x) \tag{1}$$

where f_i is the function implemented by the i-th layer and x is the input.

The intermediate activation after layer i is:

$$h_i = f_i(h_{i-1}), \quad h_0 = x \tag{2}$$

If the network is split at layer j, the DNN is partitioned as:

$$f^h(x) = f_j \circ f_{j-1} \circ \cdots \circ f_1(x) \tag{3}$$

$$f^t(h_j) = f_L \circ f_{L-1} \circ \cdots \circ f_{j+1}(h_j) \tag{4}$$

The total inference latency when splitting at layer j under batch size b and bandwidth r is:

$$T_{\text{inference}}(j; b, r) = T^h_{0,j}(b) + \frac{L_j(b)}{r} \times 1000 + T^t_{j,L}(b) \tag{5}$$

where: $T^h_{0,j}(b)$: client computation time, $T^t_{j,L}(b)$: server computation time, $L_j(b)$: size of intermediate activation at layer j.

The optimization problem is formulated as:

$$j^* = \arg\min_{j \in \mathcal{S}} T_{\text{inference}}(j; b, r) \tag{6}$$

where $\mathcal{S}$ is the set of candidate split points (typically bottleneck layers where compression $c_j = |h_j|/|x| < 1$).

3.2 Genetic Algorithm-Based Optimization

Each individual in the population encodes a candidate split point j. The fitness function is defined as:

$$\text{Fitness}(j) = \frac{1}{T_{\text{inference}}(j; b, r)} \tag{7}$$

The Genetic Algorithm (GA) process is structured as follows:

- Randomly initialize a population of size P from the candidate split set $\mathcal{S}$.
- For each of the G generations:
  - Evaluate the fitness of each individual based on the inverse of the estimated inference latency.
  - Select the elite individual with the highest fitness to carry over to the next generation.
  - Perform tournament selection to probabilistically choose parents based on fitness scores.
  - Apply one-point crossover to generate offspring from selected parents.
  - Mutate offspring with a probability μ to introduce diversity into the population.
  - Form the new population by combining the elite individual and the offspring.
- After G generations, select the individual with the highest fitness as the optimal split point for each bandwidth configuration.

Algorithm 1 Genetic Algorithm for Optimal Split Point Selection

```
function GA-Split(S, B, P, G, μ)
    x* ← ∅
    for each b_i ∈ B do
        Initialize Pop ⊂ S with P individuals
        for generation g = 1 to G do
            for each j ∈ Pop do
                Evaluate Fitness(j) = 1/T_inference(j; b_i, r_i)
            end for
            Select elite j_elite = arg max Fitness(j)
            Select parents via tournament selection
            Apply crossover to generate offspring
            Apply mutation to offspring with probability μ
            Pop ← {j_elite} ∪ offspring
        end for
        x*[b_i] ← arg max_{j∈Pop} Fitness(j)
    end for
    return x*
end function
```

3.3 Offline Profiling

Prior to optimization, profiling is conducted offline to record $T^h_{0,j}(b)$, $T^t_{j,L}(b)$, and $L_j(b)$ for all $j \in \mathcal{S}$ and batch sizes b. These metrics are stored in a lookup table to enable efficient runtime inference cost estimation.

3.4 Comparison with Dynamic Split Computing

In recent work, Dynamic Split Computing (DSC) has been proposed to optimize the split location based on pre-profiled inference costs under varying system conditions. Specifically, DSC identifies natural bottleneck layers where the activation size is compressed relative to the input, and performs exhaustive offline profiling of the head and tail inference latencies at each candidate split point across different batch sizes. At runtime, it selects the split point that minimizes the end-to-end latency by simple table lookup, without any further optimization or model retraining.

The computational complexity of DSC is $O(s \times b)$, where s denotes the number of candidate split points and b represents the number of batch sizes. Each candidate split requires independent profiling of both head-side and tail-side inference times for every batch size, which can become costly when the model has many natural bottlenecks or when multiple batch configurations must be supported. However, the runtime complexity of DSC is $O(1)$, as it merely requires a fast lookup from the precomputed batch table.

In contrast, our proposed GA-Split method first leverages the same profiling strategy to generate a lookup table, but subsequently applies a global optimization framework based on Genetic Algorithms. Rather than exhaustively evaluating all possible splits, GA-Split searches the split space using a guided evolutionary process, balancing exploration and exploitation across generations. The offline optimization complexity of GA-Split is $O(b \times G \times P)$, where G is the number of generations and P is the population size. Since fitness evaluations are performed via efficient lookup operations, the actual runtime per evaluation is negligible. The runtime complexity remains $O(1)$, similar to DSC.

Overall, while DSC relies on heuristic and greedy selection based solely on offline measurements, GA-Split provides a globally optimized solution with a manageable offline cost. Moreover, GA-Split can dynamically adapt to more complex system objectives, such as multi-criteria optimization of latency and energy—capabilities that traditional heuristic-based methods are not well-equipped to handle.

The following Table 1 summarizes the main differences between GA-Split and DSC:

4 Performance Evaluation

In this section 4, we evaluate the performance of the proposed GA-Split optimization framework through comprehensive experiments. We first introduce the experimental setup, system configurations, and evaluation metrics, followed by a detailed analysis of the experimental results compared against the original dynamic split computing baseline.

4.1 System Profiling, Optimization Strategy, and Evaluation Methodology

To evaluate the proposed GA-Split framework, we simulate a client-server split learning environment. The client represents a resource-constrained edge device, while the server simulates a powerful cloud or edge infrastructure. A pre-trained EfficientNet-B7 model is employed as the testbed, deployed entirely on a single CPU using TensorFlow OneDeviceStrategy. To emulate limited-resource conditions, the maximum number of intra-op and inter-op parallelism threads is restricted to 16, and GPU acceleration is disabled.

In the profiling phase, we record inference times for three configurations: full device execution (whole device), full server offloading (whole edge), and split computing at identified natural bottleneck layers. Only layers with substantial compression, such as `block7d_add`, are considered as candidate split points. For each candidate and batch size, the client-side (head) inference time, server-side (tail) inference time, and intermediate activation size are measured. Batch sizes from 1 to 30 are evaluated, and network bandwidths vary from 0.1 Mbps to 100 Mbps.

During the optimization phase, two split selection methods are evaluated. The original baseline method employs exhaustive heuristic search to minimize the end-to-end latency, while the proposed GA-Split method uses a genetic algorithm with a population size of 60, 400 generations, and a mutation probability of 0.2 to globally optimize the split decision.

Three key evaluation metrics are considered. The first metric is the average inference time across all batch sizes and network conditions, assessing whether GA optimization introduces any significant inference latency overhead. The second metric is the split computing utility rate, defined as the percentage of scenarios where split computing provides latency benefits compared to full device execution or full offloading. The third metric evaluates the offline computational overhead of each method, as excessive offline costs may impact the scalability of deployment.

4.2 Performance Comparison and Optimization Insights

The experimental evaluation highlights the effectiveness of the proposed GA-Split optimization framework compared to the original heuristic-based dynamic split computing baseline. In particular, the average inference time achieved by GA-Split is 27846.72 ms, which remains comparable to the baseline's inference time of 27612.13 ms. The minor increase of approximately 0.8% in latency is considered acceptable, especially when weighed against the substantial improvements in the effectiveness of split computing.

Figure 2 presents the average inference times across varying batch sizes for both GA-Split and the original baseline method. As observed, the inference times for both methods grow steadily with the batch size. Although GA-Split exhibits slightly higher latency at certain points, particularly for large batch sizes, the overall trend remains closely aligned with the baseline, confirming that GA-based optimization does not introduce significant inference overhead.

A key advantage of GA-Split lies in its significantly higher split computing utility rate. As shown in Figure 3, GA-Split consistently selects bottleneck layers as optimal split points across a wider range of batch sizes compared to the baseline. In particular, for almost all batch sizes, GA-Split identifies more bottleneck opportunities where split computing is advantageous. This result directly correlates with the overall improvement in split computing utility rate, with GA-Split achieving 74.38% compared to 28.70% by the original method.

Further insights are illustrated in Figure 4, where the split decisions for GA-Split and the original method are visualized across different bandwidth and batch size configurations. In GA-Split (left),

Aspect	Dynamic Split Computing	GA-Split (Proposed)
Offline Complexity	$O(s \times b)$	$O(b \times G \times P)$
Runtime Complexity	$O(1)$	$O(1)$
Optimization Type	Heuristic, lookup-based	Global search, evolutionary
Adaptivity	Limited (fixed lookup)	Flexible, bandwidth-aware
Scalability	Degrades with s large	Tunable via P, G
Profiling Overhead	High if many splits	Moderate, depends on P, G

Table 1: Comparison between DSC and GA-Split.

the decisions are more consistently adaptive, favoring dynamic switching between full edge execution and split computing based on system conditions. In contrast, the original method (right) often exhibits rigid split selections, failing to adapt efficiently to changes in network bandwidth or batch sizes. This dynamic adaptability of GA-Split further substantiates its superior performance under practical, fluctuating network environments.

Moreover, although GA-Split incurs a higher offline computation cost due to the genetic search process, this overhead occurs only once during the system profiling phase. Importantly, both GA-Split and the baseline approach rely on lightweight lookup operations at runtime, resulting in negligible runtime overhead.

Overall, these results validate that GA-Split provides a globally optimized split strategy that significantly enhances the practicality and benefits of split computing, while maintaining inference latency comparable to traditional heuristics. The detailed comparative results are summarized in Table 2.

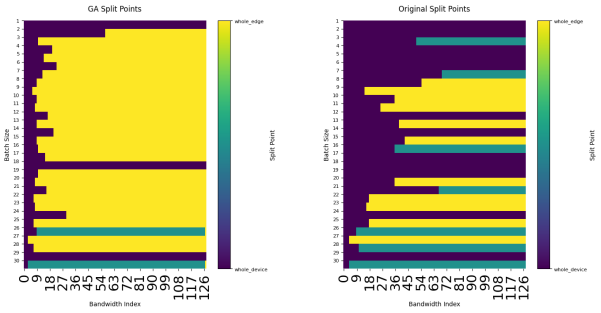


Figure 2: Average inference time across batch sizes for GA-Split and original baseline.

Finally, the experimental results confirm that the proposed GA-Split framework successfully enhances split computing adaptability and optimization without compromising inference efficiency. By leveraging global optimization, GA-Split significantly increases the split computing utility rate and enables dynamic adaptation to varying bandwidth and computational conditions. Our findings demonstrate the practicality and robustness of the GA-Split approach for real-world edge AI deployments.

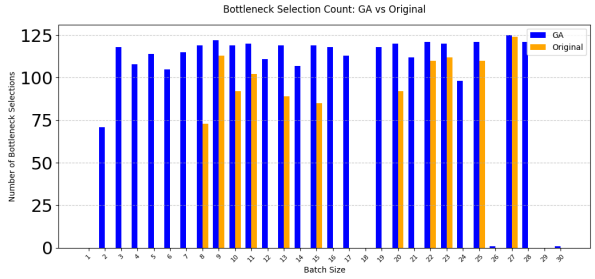


Figure 3: Comparison of bottleneck layer selection counts between GA-Split and original baseline across batch sizes.

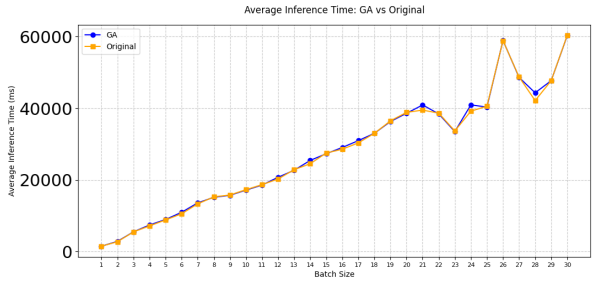


Figure 4: Visualization of selected split points across different bandwidths and batch sizes: (Left) GA-Split, (Right) Original baseline.

5 Conclusions

In this paper, we proposed GA-Split, a novel genetic algorithm-based framework for dynamic split point optimization in split learning systems. By formulating the split selection as a latency minimization problem and leveraging the global search capabilities of genetic algorithms, GA-Split significantly improves the applicability and efficiency of split computing in resource-constrained edge environments. Extensive experiments on large-scale models, such as EfficientNet-B7, demonstrated that GA-Split achieves a substantially higher split computing utility rate while maintaining comparable inference latency to heuristic-based baselines. Furthermore, GA-Split exhibits strong adaptability to varying bandwidths and system conditions, confirming its practical effectiveness for real-world deployments. Future work will focus on extending GA-Split to multi-objective optimization, considering factors such as energy

Metric	Original (Baseline)	GA-Split (Proposed)
Average Inference Time (ms)	27612.13	27846.72
Split Computing Utility Rate (%)	28.70%	74.38%
Offline Computation Cost	Low (profiling only)	Moderate (profiling + GA search)
Runtime Overhead	Negligible	Negligible
Optimization Method	Heuristic Lookup	Genetic Algorithm (Global Search)
Adaptivity to Bandwidth Changes	Limited	High
Scalability with Model Size	Moderate	Better (Population-based search)

Table 2: Detailed comparison between DSC and GA-Split optimization.

consumption, privacy preservation, and asynchronous client-server dynamics.

6 Acknowledgments

This research is funded by University of Economics Ho Chi Minh City (UEH), Vietnam.

References

[1] Arian Bakhtiarnia, Nemanja Milosevic, Qi Zhang, Dragana Bajovic, and Alexandros Iosifidis. 2021. Dynamic Split Computing for Efficient Deep Edge Intelligence. In *Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 993–1003.

[2] Xing Chen, Jingtao Li, and Chaitali Chakrabarti. 2022. Energy and Loss-Aware Selective Updating for SplitFed Learning with Energy Harvesting-Powered Devices. *Journal of Signal Processing Systems* 94 (2022), 961–975.

[3] Ayush Chopra, Surya Kant Sahu, Abhishek Singh, Abhinav Java, Praneeth Vepakomma, Vivek Sharma, and Ramesh Raskar. 2021. Adasplit: Adaptive trade-offs for resource-constrained distributed deep learning. *arXiv preprint arXiv:2112.01637* (2021).

[4] David E. Goldberg. 1989. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

[5] Praveen Joshi, Xiaowen Zhang, Pratiksha Ray, et al. 2022. Performance and Information Leakage in Splitfed Learning and Multi-Head Split Learning in Healthcare Data and Beyond. *Methods and Protocols* 5, 4 (2022), 60.

[6] Tran Anh Khoa, Minh-Son Dao, Do-Van Nguyen, and Koji Zettsu. 2024. Clustering-Enhanced Reinforcement Learning for Adaptive Offloading in Resource-Constrained Devices. In *2024 IEEE International Conference on Smart Computing (SMARTCOMP)*. 133–140. doi:10.1109/SMARTCOMP61445.2024.00039

[7] Tran Anh Khoa, Do-Van Nguyen, Minh-Son Dao, and Koji Zettsu. 2021. Fed xData: A Federated Learning Framework for Enabling Contextual Health Monitoring in a Cloud-Edge Network. In *2021 IEEE International Conference on Big Data (Big Data)*. 4979–4988. doi:10.1109/BigData52589.2021.9671536

[8] Tran Anh Khoa, Nguyen Dang Trac, Vo Phuc Tinh, Nguyen Hoang Nam, Duc Ngoc Minh Dang, Hoang Hai Son, and Pham Duc Lam. 2024. Safety Is Our Friend: A Federated Learning Framework Toward Driver's State and Behavior Detection. *IEEE Transactions on Computational Social Systems* 11, 2 (2024), 2340–2358. doi:10.1109/TCSS.2023.3273727

[9] Zhuohang Li, Chao Yan, Xinmeng Zhang, Gharib Gharibi, Zhijun Yin, Xiaoqian Jiang, and Bradley A Malin. 2024. Split learning for distributed collaborative training of deep learning models in health informatics. In *AMIA Annual Symposium Proceedings*, Vol. 2023. 1047.

[10] Jun Liu, Xiang Lin, et al. 2022. Split Learning for Wireless Edge Networks: Principles, Challenges, and Future Directions. *IEEE Wireless Communications* 29, 5 (2022), 112–118.

[11] Yoshitomo Matsubara, Marco Levorato, and Francesco Restuccia. 2021. Split Computing and Early Exiting for Deep Learning Applications: Survey and Research Challenges. *ACM Computing Surveys (CSUR)* 54, 8 (2021), 1–37.

[12] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)* (2017), 1273–1282.

[13] Mingxing Tan and Quoc Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*. PMLR, 6105–6114.

[14] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. 2018. Split learning for health: Distributed deep learning without sharing raw patient data. *Proceedings of the 2018 AAAI Workshop on Privacy-Preserving Artificial Intelligence (PPAI)* (2018).

[15] Hao Wang, Kuan-Ching Lin, Xiaoyan Liang, and Weisong Wu. 2021. FedAdapt: Adaptive Offloading for IoT Devices in Federated Learning. In *Proceedings of the 40th IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1484–1494.

[16] Danyang Xiao, Chengang Yang, and Weigang Wu. 2022. Mixing Activations and Labels in Distributed Training for Split Learning. *IEEE Transactions on Parallel and Distributed Systems* 33, 11 (2022), 3165–3177.

[17] Danyang Xiao, Chengang Yang, and Weigang Wu. 2022. Mixing Activations and Labels in Distributed Training for Split Learning. *IEEE Transactions on Parallel and Distributed Systems* 33, 11 (2022), 3165–3177.